Quantitative Trading in Prediction Markets: Architecting Algorithmic Edges Across Kalshi and Polymarket

The maturation of prediction markets from niche, speculative platforms into heavily capitalized, regulated financial exchanges has birthed a new paradigm in quantitative finance. By early 2026, the global prediction market industry was processing over twenty billion dollars in monthly transaction volume, driven by the convergence of institutional liquidity, high-frequency trading infrastructure, and the widespread adoption of Central Limit Order Book (CLOB) architectures[1]. The landscape is currently dominated by two primary venues operating under the regulatory oversight of the Commodity Futures Trading Commission (CFTC): Kalshi, and Polymarket US, operating under the entity QCX LLC[3].

Finding reproducible, profitable edges in this ecosystem requires far more than discretionary forecasting. It demands a synthesis of market microstructure analysis, highly optimized data engineering, rigorous statistical modeling, and low-latency execution architecture. The following analysis explores the structural mechanics of these platforms, the specific API and data handling techniques required for robust integration, and the quantitative strategies deployed by modern algorithmic trading bots to extract consistent alpha from predictive inefficiencies.

Market Microstructure and Regulatory Frameworks

Understanding the operational mechanics and regulatory constraints of Kalshi and Polymarket is a fundamental prerequisite for developing automated trading systems. Both platforms facilitate the trading of event contracts, which are binary derivatives that resolve to either a full payout (typically $1.00) or zero based on the outcome of a specified real-world event[1]. However, their underlying architectures, fee structures, and legal classifications dictate highly specific algorithmic approaches.

Kalshi operates as a federally regulated Designated Contract Market (DCM), offering direct access across the United States for event contracts spanning economics, politics, weather, and sports[1]. The exchange utilizes a traditional CLOB model, matching buyers and sellers neutrally without taking directional exposure to the underlying events[7]. A critical component of the Kalshi microstructure is its dynamic fee model, which algorithms must rigorously incorporate into expected value calculations.

In contrast, the Polymarket ecosystem is bifurcated into Polymarket Global (the crypto-native, international platform) and Polymarket US. Polymarket US was established following the one hundred and twelve million dollar acquisition of QCX LLC and QC Clearing LLC, granting the platform CFTC-licensed derivatives exchange and clearinghouse status[3]. While Polymarket Global operates using the USDC stablecoin on the Polygon blockchain, Polymarket US requires full Know Your Customer (KYC) verification, settles in USD via approved Futures Commission

Merchants (FCMs), and operates entirely within the strictures of U.S. financial law[6].
The regulatory status of these platforms has profound implications for quantitative traders, particularly regarding taxation. The passage of the One Big Beautiful Bill Act (OBBBA) in 2025 capped gambling loss deductions at ninety percent of gambling winnings starting with the 2026 tax year[11]. However, because Polymarket US and Kalshi operate as CFTC-regulated DCMs, their contracts present a strong case for classification under Section 1256 of the tax code, fundamentally altering the post-tax profitability metrics for algorithmic systems[11].

Feature	Kalshi	Polymarket US (QCX)
Regulatory Status	CFTC-Regulated DCM	CFTC-Regulated DCM
Order Book Structure	Central Limit Order Book (CLOB)	Central Limit Order Book (CLOB)
Settlement Asset	USD	USD (via FCMs)
Resolution Mechanism	Centralized Exchange Authority	UMA Optimistic Oracle / DVM
Primary Interfaces	REST, WebSocket, FIX	REST, gRPC, WebSocket, FIX

Capitalizing on Microstructural Fee Dynamics

To locate a mathematical edge on Kalshi, algorithms must first navigate its complex fee structure. Kalshi employs a probability-weighted maker-taker fee schedule[1]. The fee is not a flat rate; it scales parabolically based on the contract's implied probability, ensuring the exchange monetizes activity across the entire probability spectrum. The taker fee is calculated dynamically, rounding up to the nearest centicent, using a specific multiplier against the contract's price and its inverse probability.

$$\text{Taker Fee} = \lceil 0.07 \times C \times P \times (1 - P) \rceil$$

Where C represents the number of contracts being traded and P represents the price of the contract in dollars (e.g., fifty cents is expressed as 0.5)[12]. Consequently, a contract priced at $0.50 yields the maximum absolute fee per contract of $0.0175, representing 3.5 percent of the capital committed[13]. However, for extreme probabilities, such as a contract priced at $0.05, the fee drops to $0.0034 per contract[12]. Maker fees follow an identical parabolic curve but

utilize a significantly lower multiplier of 0.0175, heavily incentivizing liquidity provision[12].
This fee structure induces specific, exploitable market anomalies. Academic analysis of Kalshi transaction data indicates that takers frequently suffer from a "winner's curse," particularly on low-priced, low-probability contracts[13]. The forecast errors combined with the heavy percentage drag of taker fees mathematically necessitate aggressive win rates to achieve long-term profitability when crossing the spread on contracts priced below ten cents[13]. Algorithmic systems are therefore engineered to exclusively utilize limit orders (maker fees) whenever possible, effectively forcing impatient retail participants to absorb the higher taker fees[13].
Polymarket US approaches fees differently, employing a dynamic taker fee model based on the underlying category of the market rather than a strict probability curve. The fee peaks at 1.80 percent for cryptocurrency-related markets, drops to 0.75 percent for sports markets, and remains completely fee-free for geopolitical and world events[6]. This category-based structure dictates capital allocation, driving algorithmic arbitrageurs toward the fee-free geopolitical categories where much tighter spreads can be captured without the drag of transaction costs[6].

Data Engineering: Partitioning and Historical Analysis

Generating consistent alpha requires immaculate data engineering and historical backtesting. Kalshi partitions its exchange data into live and historical tiers to maintain low latency on its primary servers as the volume of settled markets expands[15]. Systems querying the REST API for backtesting purposes must navigate this architecture carefully.
The boundary between live and historical data is defined by a set of dynamic cutoff timestamps. An automated system must first call the GET /historical/cutoff endpoint to retrieve these precise timestamps[15]. If the algorithm requires data older than the cutoff, it must route queries away from standard live endpoints and toward dedicated historical endpoints.

Partition Field	Meaning	Associated Historical Endpoint
market_settled_ts	Market settlement time	GET /historical/markets
trades_created_ts	Trade fill time	GET /historical/trades
orders_updated_ts	Order cancellation or execution time	GET /historical/orders

Algorithms designed to build a complete order and fill history must query both the live and historical endpoints and programmatically merge the resulting JSON payloads[15]. For comprehensive candlestick analysis, which is vital for mean-reversion and momentum

strategies, bots rely on the GET /historical/markets/{ticker}/candlesticks endpoint, while utilizing pagination cursors to retrieve massive datasets limited to one thousand results per request[15]. This strict data hygiene is the foundational step for discovering statistical edges, as models trained exclusively on the live GET /markets endpoint will suffer from survivorship bias, failing to account for recently settled contracts that have crossed the market_settled_ts threshold[15].

Kalshi Bot Architecture and Real-Time Integration

For live trading, Kalshi relies heavily on WebSockets to deliver order book updates, trade executions, and fill notifications with minimal latency. The primary WebSocket endpoint for modern integrations is located at wss://external-api-ws.kalshi.com/trade-api/ws/v2[17]. Establishing this connection requires explicit authentication executed during the initial connection handshake. The bot must generate a cryptographic signature using an API key ID and a private key. The signature string is constructed by concatenating the current Unix timestamp in milliseconds, the HTTP method "GET", and the connection path "/trade-api/ws/v2"[18]. These elements are passed into the connection headers as KALSHI-ACCESS-KEY, KALSHI-ACCESS-SIGNATURE, and KALSHI-ACCESS-TIMESTAMP[18].
Once authenticated, the architecture must manage channel subscriptions via standardized JSON command payloads. Bots typically issue a subscribe command targeting the orderbook_delta channel to receive Layer 2 book updates, alongside the ticker and fill channels for price feeds and private execution notifications[18]. A robust quantitative architecture does not rely solely on incremental updates; it must first cache the initial order book state via a REST snapshot before asynchronously applying WebSocket deltas, ensuring a millisecond-accurate reflection of the exchange's matching engine[18].
Production-grade systems must also implement resilient error handling and connection lifecycles. The WebSocket protocol returns specific error codes that the bot must parse and resolve autonomously. For instance, Code 1 indicates an invalid JSON payload or schema mismatch, Code 6 indicates an attempt to subscribe to an already active channel, and Code 9 signals an authentication failure[18]. If a network interruption occurs, the bot must implement automatic reconnection logic utilizing exponential backoff while simultaneously clearing any internal stale state to prevent executing trades based on outdated probabilities[18].
Furthermore, a bot's capacity to scale on Kalshi is directly tied to its trading volume. Kalshi enforces API rate limits through a tiered system. Once a day, the exchange reviews the account's trailing thirty-day volume share, defined as the account's volume divided by twice the previous calendar month's total exchange volume[20]. Crossing specific volume thresholds elevates the account through the Expert, Premier, Paragon, Prime, and Prestige tiers, which exponentially increases the permitted number of active WebSocket connections and REST request bursts[20].

Polymarket Bot Architecture and Real-Time Integration

The Polymarket US infrastructure is engineered for institutional scaling, fundamentally

separating public reference data from authenticated trading execution. The architecture divides access between the public Gateway API (gateway.polymarket.us) for unauthenticated queries such as order books and market metadata, and the authenticated Trading API (api.polymarket.us) for order placement and portfolio management[22].

Modern algorithmic deployments on Polymarket typically eschew raw HTTP requests in favor of official Software Development Kits (SDKs). The polymarket-us Python SDK and its TypeScript equivalent seamlessly handle Private Key JSON Web Token (JWT) authentication, request signing, and provide fully typed interfaces for all endpoints[23]. Before generating API keys, developers must complete full identity verification through the Polymarket US application, ensuring compliance with CFTC regulations[23].

For the highest tier of algorithmic execution, polling REST endpoints is wholly insufficient due to firm-level rate limits of one hundred requests per second[24]. Sophisticated architectures bypass REST entirely for market data, utilizing Polymarket's gRPC Streaming APIs located at grpc-prod.polymarketexchange.com:443[24]. gRPC offers heavily compressed data payloads and substantially lower latency compared to standard REST or WebSockets, avoiding rate-limiting bottlenecks and reducing the overall infrastructure footprint[24]. For institutions requiring private connectivity that bypasses the public internet entirely, Polymarket supports AWS PrivateLink, allowing virtual private clouds to connect directly to the exchange's Financial Information eXchange (FIX) API[25].

Polymarket Exchange API environments are strictly segregated to facilitate safe algorithmic development. Bots are initially deployed against the Development environment (api.dev01.polymarketexchange.com), advanced to Pre-production (api.preprod.polymarketexchange.com) for integration testing, and finally released into Production[24]. Across all environments, access tokens generated via Auth0 domain endpoints expire every three minutes, necessitating an internal background thread within the bot dedicated exclusively to automatic token refreshing[24].

Execution management on Polymarket requires parsing synchronous order responses to determine state changes. Bots monitor execution types such as EXECUTION_TYPE_NEW for working order confirmations, EXECUTION_TYPE_PARTIAL_FILL, and EXECUTION_TYPE_REJECTED[26]. Prices submitted through the API must be represented as int64 values, calculated by dividing the desired decimal price by the instrument's specific price_scale parameter, which is cached during the initial reference data ingestion[24].

A powerful open-source alternative to the official SDKs is NautilusTrader, a high-performance, thread-safe trading system written in Rust with Python bindings[27]. NautilusTrader normalizes the CLOB APIs of Polymarket, treating the binary outcome tokens as standardized exotic options[28]. This system natively handles EIP-712 order signing for externally owned accounts (Signature Type 0) and proxy wallets (Signature Types 1 and 2), while managing the necessary Conditional Token Framework (CTF) contract allowances required to authorize the exchange to move USDC collateral[28]. The integration allows the routing of up to fifteen independent limit orders through a single batched HTTP POST request, drastically improving execution throughput during volatile market events[30].

Algorithmic Strategies: Extracting the Statistical Edge

Generating consistent alpha in prediction markets requires moving beyond subjective forecasting and deploying algorithmic strategies that exploit structural, logical, and latency-based inefficiencies. The most consistently profitable approaches rely on computational speed, mathematical arbitrage, and automated liquidity provision.

Cross-Exchange Arbitrage and Hedging

The most direct form of algorithmic edge is low-risk spread capture across competing platforms. Because Polymarket and Kalshi host distinct liquidity pools with separate user bases—Polymarket attracting a blend of crypto-native participants and institutional flow, while Kalshi attracts retail and macroeconomic speculators—the implied probabilities of identical real-world events frequently diverge[31].

An arbitrage bot continuously monitors both platforms, mapping the metadata of events to ensure perfect parity. If Kalshi prices the YES contract for an inflation metric at sixty-three cents, while Polymarket prices the identical YES contract at fifty-seven cents, a pricing gap exists[32]. The bot executes a simultaneous trade: buying the YES contract on Polymarket and hedging the position by selling the equivalent contract on Kalshi, locking in a six-cent gross spread[32].

Realizing this profit requires the calculation of the Net Expected Value matrix, accounting for transaction costs across both venues[32]. The bot must normalize decimal odds, fractional shares, and dynamic fee schemas instantly to ensure the net margin exceeds the configured threshold[31]. This strategy carries two primary risks. First is legging risk, where one side of the trade fills while the other slips or is cancelled, leaving the bot directionally exposed[32]. Second is settlement mismatch, occurring if the exact wording or resolution oracle differs slightly between the two platforms, potentially resulting in a catastrophic scenario where both contracts resolve to zero. Advanced algorithms utilize natural language processing to confirm exact contract parity before authorizing execution[32].

Automated Market Making (AMM) on Central Limit Order Books

Given the retail tendency to make aggressive, directional bets on outcomes, market making remains the most consistent algorithmic strategy, boasting historical win rates between 78 and 85 percent[33]. Rather than predicting the actual outcome of the event, the bot acts as a neutral liquidity provider, submitting both bids and asks on the order book to capture the bid-ask spread[1].

If the statistical fair value of a contract is sixty cents, the bot places a resting bid at fifty-eight cents and a resting ask at sixty-two cents. When impatient retail flow crosses the spread, the bot captures four cents per round-trip trade, representing a massive annualized return on capital[33]. Because Kalshi charges reduced fees for maker limit orders, and Polymarket relies on deep CLOBs for price discovery, this strategy is highly incentivized by the structural design of the exchanges[1].

The primary danger in market making is adverse selection, which occurs when trading against a

counterparty possessing superior information[33]. If breaking news occurs, the fair value instantly shifts, and the bot's resting limit orders will be run over by informed traders before they can be cancelled. Consequently, market-making algorithms must dynamically adjust spread widths based on real-time order book volatility, pull liquidity entirely immediately preceding scheduled data releases (such as Federal Reserve announcements or CPI prints), and enforce strict inventory limits to prevent the accumulation of concentrated directional risk[33].

Logical and Correlation Arbitrage

Prediction markets frequently list derivative or correlated markets that suffer from internal mathematical contradictions. Logical arbitrage exploits these mathematically impossible pricing structures[33]. A sophisticated bot utilizes graph theory to continuously map probability dependencies across thousands of contracts simultaneously.

For instance, if a market predicting a specific candidate will win an election trades at an implied probability of thirty-five percent, but a broader market predicting that candidate's political party will win trades at thirty-two percent, a mathematical violation has occurred[33]. The subset cannot possess a higher probability than the superset. The bot immediately buys the underpriced broader market and sells the overpriced specific market to capture the guaranteed differential.

Similarly, cumulative probability violations occur when mutually exclusive outcomes sum to a value greater than or less than one hundred percent (excluding exchange fees). If the sum of all possible outcomes in a given market totals one hundred and eleven percent, the algorithm systematically sells the overpriced contracts across the board, locking in an arbitrage profit regardless of the ultimate resolution[33]. This edge persists because human traders suffer from severe cognitive overload when tracking hundreds of micro-markets, whereas graph-based bots identify contradictions in milliseconds[33].

The Information Edge: AI-Powered Probability Updating

News latency arbitrage leverages the time delay between a real-world event occurring and the prediction market fully absorbing and pricing the information. In traditional financial equities, this window lasts fractions of a millisecond; in prediction markets, the inefficiency window can range from thirty seconds to five minutes due to the retail-heavy nature of the participant base[31].

Modern quantitative architectures ingest raw data feeds directly from official news wires, social media, and on-chain metrics, feeding them into ensemble Artificial Intelligence models running on high-speed inference servers[33]. The AI performs rapid Bayesian inference to calculate an updated probability[33]. If a key witness in a political trial unexpectedly recants testimony, the AI parses the sentiment, cross-references the credibility of the reporting sources, and calculates that the probability of a specific outcome has shifted by fifteen percent[33]. If the CLOB still reflects the stale probability, the bot executes aggressive taker orders to absorb the mispriced liquidity before human traders can manually update their resting limit orders[33].

Fading Retail Overreactions and Mean Reversion

Retail traders in prediction markets frequently exhibit severe behavioral biases, particularly

during live sporting events or volatile election nights. Emotional trading leads to dramatic overreactions to short-term variance, causing contract probabilities to swing far beyond their statistical fair value[34].

Quantitative bots identify these overreactions by comparing live state data—such as time remaining, score differentials, and possession metrics during an NFL game—against massive databases of historical outcomes[34]. If historical data indicates a team still holds a forty percent chance of victory, but the prediction market has crashed the contract to twenty cents due to panic selling following a turnover, the bot aggressively buys the undervalued asset[34]. This strategy does not hold the contract to expiration; rather, it trades the mean reversion, selling the contract back into the market once the price stabilizes, effectively fading the emotional extremes of retail participants to capture the spread[34].

Infrastructure Optimization: The Proximity Advantage

To execute high-frequency momentum strategies and latency arbitrage effectively, physical proximity to the exchange matching engines is mandatory. The optimal deployment strategy for an automated trading bot involves leasing an ultra-low latency Virtual Private Server (VPS)[10]. For operations targeting Polymarket Global, which historically hosts its primary Gamma and CLOB API infrastructure in Amazon Web Services (AWS) eu-west-2 located in London, developers frequently stage infrastructure in Amsterdam or Dublin[8]. This strategic placement avoids regional restrictions while achieving single-digit millisecond round-trip times to AWS backbones[8]. For Kalshi and the regulated Polymarket US platform, deployments in New York or Northern Virginia (AWS us-east-1) are standard[10]. Utilizing hardware equipped with high clock-speed processors, such as the AMD Ryzen 9 9950X, paired with high-speed DDR5 RAM and NVMe storage, ensures that the bot's internal spread calculations and JSON parsing introduce virtually zero computational lag into the execution pipeline[35].

Oracle Risk and Decentralized Resolution Vulnerabilities

While the edges available in prediction markets are lucrative, the asset class harbors structural vulnerabilities distinct from traditional equities. Quantitative strategies must mathematically discount these risks, particularly regarding market resolution. Kalshi mitigates this by acting as a centralized exchange authority, settling markets according to strictly defined, pre-published criteria under CFTC oversight[7].

Polymarket, however, relies entirely on the UMA Optimistic Oracle for dispute resolution, introducing unique systemic game-theory risks[38]. When a Polymarket market closes, an asserter proposes a resolution outcome and posts a $750 USDC bond[38]. This action initiates a two-hour liveness period[38]. If the proposal remains undisputed, the market resolves, and the asserter earns a protocol reward[38]. However, any actor can dispute the claim by posting a matching $750 bond[40].

Disputed claims escalate to the Data Verification Mechanism (DVM), which coordinates a

decentralized, stake-weighted vote by UMA token holders[40]. The DVM operates on a commit-and-reveal scheme, where voters submit hidden cryptographic hashes of their votes for twenty-four hours before revealing them, preventing participants from simply copying the visible majority[42]. Voters reference specific UMA Improvement Proposals (UMIPs) to determine the methodology for deriving the correct outcome from off-chain sources[40]. Accurate voters are rewarded with protocol emissions, while inaccurate or absent voters are slashed, losing a portion of their staked tokens[41].

While designed for decentralized truth-seeking, the DVM carries a severe structural flaw: if a large token holder, or a coordinated cartel, possesses a massive financial stake in a specific Polymarket outcome, the potential profit from manipulating the vote could mathematically eclipse the loss in value of their UMA holdings resulting from a loss of protocol credibility[43]. Historical on-chain analysis indicates that up to sixty percent of active UMA voters can be linked to live Polymarket accounts, establishing clear conflict-of-interest vectors[43]. Automated bots designed to hold contracts to expiration must incorporate a "manipulation discount" into their Kelly Criterion sizing when trading on Polymarket, compensating for the non-zero probability that the decentralized oracle is corrupted by participants holding highly asymmetric financial incentives[43].

Strategic Synthesis and Future Outlook

The evolution of prediction markets into highly regulated financial venues featuring Central Limit Order Books has established a fertile environment for algorithmic trading. The inherent divergence in user demographics, category-based fee models, and technological infrastructure between Kalshi and Polymarket creates persistent, mathematically exploitable structural inefficiencies.

By deploying robust data architectures that leverage gRPC streaming, WebSocket deltas, and private FIX execution channels, quantitative systems can extract alpha through cross-exchange arbitrage, automated market making, logical correlation mapping, and AI-driven news latency modeling. However, achieving reproducible profitability requires meticulous attention to microstructural constraints. An algorithmic bot must flawlessly navigate Kalshi's parabolic maker-taker fee algorithms to avoid the mathematical certainty of the winner's curse on low-probability trades. It must normalize APIs between fiat-based regulated exchanges and crypto-native oracle-dependent platforms, standardizing data handling across different settlement models. Finally, it must enforce draconian risk management protocols, utilizing exponential backoff during disconnects, halting trading during balance fragmentation, and actively discounting the unique dangers of decentralized resolution mechanisms. As trading volumes continue to scale and institutional participants arrive en masse, the window for simple latency arbitrage will inevitably narrow, transferring the dominant algorithmic edge to operators possessing superior probability models, advanced graph-theory integration, and precision market-making infrastructure.

Works cited

1. Kalshi Stock: $22B Valuation — Is It a Buy? | TSG Invest, https://tsginvest.com/kalshi/
2. How Prediction Markets Scaled to USD 21B in Monthly Volume in 2026 | TRM Labs, https://www.trmlabs.com/resources/blog/how-prediction-markets-scaled-to-usd-21b-in-monthly-volume-in-2026
3. Prediction markets in 2026: Key trends reshaping forecasting, trading, and regulation, https://metamask.io/news/prediction-market-overview-trends-2026
4. POLYMARKET US RULEBOOK, https://polymarketexchange.com/files/legal/Polymarket%20US%20Rulebook%20(2026.07.02).pdf
5. Polymarket - Decentralized Prediction Market | BUVCG Research - Medium, https://medium.com/buvcg-research/polymarket-deep-dive-06afa8c9a02b
6. Best Prediction Market Platforms in 2026: Compare Kalshi, Polymarket & More - Bleap, https://www.bleap.finance/en-us/blog/best-prediction-market-platforms
7. How Does Kalshi Make Money and How Do Users Profit From Prediction Markets? - TMCnet, https://www.tmcnet.com/topics/articles/2026/04/28/463549-how-does-kalshi-make-money-how-users-profit.htm
8. Polymarket - Grokipedia, https://grokipedia.com/page/polymarket
9. Is Polymarket Legal in 2026? US States & Countries - DropsTab, https://news.dropstab.com/research/is-polymarket-legal
10. Polymarket US (2026 Complete Guide): Features, API Access - TradingVPS, https://tradingvps.io/polymarket-us-guide/
11. Polymarket Tax Guide 2026: How to Pay Less Tax | CountDeFi, https://www.countdefi.com/blog/polymarket-taxes
12. Kalshi Fee Schedule (PDF), https://kalshi.com/docs/kalshi-fee-schedule.pdf
13. Makers and Takers: The Economics of the Kalshi Prediction Market - Karl Whelan, https://www.karlwhelan.com/Presentations/Whelan-NYU.pdf
14. How Polymarket Makes Money: Revenue Model 2026 - Idea Usher, https://ideausher.com/blog/how-polymarket-makes-money/
15. Historical Data - API Documentation - Kalshi's API, https://docs.kalshi.com/getting_started/historical_data
16. Get Historical Trades - API Documentation, https://docs.kalshi.com/api-reference/historical/get-historical-trades
17. WebSocket API - API Documentation - Kalshi's API, https://docs.kalshi.com/websockets
18. Quick Start: WebSockets - API Documentation, https://docs.kalshi.com/getting_started/quick_start_websockets
19. WebSocket Connection - API Documentation, https://docs.kalshi.com/websockets/websocket-connection
20. Rate Limits and Tiers - API Documentation, https://docs.kalshi.com/getting_started/rate_limits
21. API Changelog - API Documentation, https://docs.kalshi.com/changelog
22. Introduction - Polymarket US Documentation, https://docs.polymarket.us/api-reference/introduction

23. Introduction - Polymarket US Documentation, https://docs.polymarket.us/api-reference/sdks/introduction
24. REST API Overview - Polymarket US Documentation, https://docs.polymarket.us/institutional/introduction
25. Environments - Polymarket US Documentation, https://docs.polymarket.us/trader-guide/environments
26. Orders API Overview - Polymarket US Documentation, https://docs.polymarket.us/api-reference/orders/overview
27. clob · GitHub Topics, https://github.com/topics/clob
28. Polymarket Integration - NautilusTrader, https://nautilustrader.io/polymarket/
29. Polymarket | AI Quantitative Trading Platform | AI量化交易平台, https://gitbookhub.com/docs/nautilustrader/integrations/polymarket
30. Polymarket - NautilusTrader, https://nautilustrader.io/docs/latest/integrations/polymarket/
31. Polymarket API & Kalshi API: Python Guide to Prediction Market Data | OddsPapi Blog, https://oddspapi.io/blog/polymarket-api-kalshi-api-vs-sportsbooks-the-developers-guide/
32. https://tradingvps.io/polymarket-vs-kalshi-arbitrage-trading-bot/
33. https://medium.com/illumination/beyond-simple-arbitrage-4-polymarket-strategies-bots-actually-profit-from-in-2026-ddacc92c5b4f
34. Built a SaaS for finding short-term mispricings in live prediction markets (NFL only so far), https://www.reddit.com/r/SaaS/comments/1q83zg3/built_a_saas_for_finding_shortterm_mispricings_in/
35. TradingVPS: High-Performance, Low Latency VPS for Trading, https://tradingvps.io/
36. Best VPS Locations for Polymarket Trading Speed, Latency & Compliance Guide.pdf, https://www.slideshare.net/slideshow/best-vps-locations-for-polymarket-trading-speed-latency-compliance-guide-pdf/287121630
37. EXHIBIT A, https://business.cch.com/srd/SCGRv25-cv-128591-1.pdf
38. How Polymarket Resolves & Pays Out: The 2-Hour Rule Explained, https://startpolymarket.com/learn/how-markets-resolve/
39. How Does Polymarket Work? A Complete Guide to the Prediction Market Platform - Bleap, https://www.bleap.finance/en-us/blog/how-does-polymarket-work
40. How does UMA's Oracle work? | UMA Documentation, https://docs.uma.xyz/protocol-overview/how-does-umas-oracle-work
41. Inside UMA Oracle | How Prediction Markets Resolution Works - Rock'n'Block, https://rocknblock.io/blog/how-prediction-markets-resolution-works-uma-optimistic-oracle-polymarket
42. UMA Protocol: How does the popular Optimistic Oracle work? - MetaLamp, https://metalamp.io/magazine/article/uma-protocol-how-does-the-popular-optimistic-oracle-work

43. How Prediction Markets Resolve: UMA Oracle Explained | Bitget News, https://www.bitget.com/news/detail/12560605489048
44. Why Is Polymarket's UMA Controversial? - Webopedia, https://www.webopedia.com/crypto/learn/polymarkets-uma-oracle-controversy/